JVD Internal Management System

System Audit & Professionalization Plan · v1.0 FINAL · 2026-07-04

JVD Internal Management System — Full System Audit & Professionalization Plan

Date: 2026-07-04 · **Branch audited:** `Emman` · **Method:** static scan of all code + full backend test run + frontend type-check

> **This document set is FINAL (v1.0).** Execution checklist: `IMPLEMENTATION_ROADMAP.md` (start there). Design system: `DESIGN_DIRECTION.md`. Feature/architecture specs (colors, notifications, email, documents, workflows, roles, dashboards, sales catalog, data integrity): `PRODUCT_IMPROVEMENT_SPEC.md`.

This document answers four questions, straightforwardly:

1. What does the system have and do?
2. What works as intended, and what doesn't?
3. What professional standards are missing?
4. What is the plan to get there?

---

1. Executive Summary

Verdict: the system is functionally rich and salvageable. Do NOT rewrite from scratch — refactor incrementally.

The bones are better than "vibe coded" suggests: Laravel 13 + React 19 is a modern, industry-standard stack; there are 126 backend tests (125 passing); RBAC, 2FA, audit logging, and rate limiting on auth exist; earlier QA security findings (RCE via upload, privilege escalation, 2FA brute-force) were verified fixed in the current code.

The gaps are almost entirely in **engineering process and operations**, not in the framework choice:

- **Zero DevOps**: no CI/CD, no Docker, no staging environment, no error monitoring, no backup strategy, no deployment pipeline. Everything is "runs on someone's laptop."
- **Zero frontend tests** and no code splitting — every one of ~42,000 lines of page code ships in one JavaScript bundle.
- **Repo hygiene is poor**: debug scripts (`test.php` – `test7.php`), a zipped PHP runtime (`php83.zip`), `rr.exe`, diff dumps, and backup files are committed to git.
- **God files**: 2,685-line React pages and 838-line controllers that no one can safely modify.
- **No branch/PR discipline**: per-person branches (`Emman`, `Val`, `Greg`) pushed directly, no code review gate.

On Kubernetes: **you do not need it, and adopting it would be a mistake at your scale.** See §5.3 for the honest sizing. Docker + a CI/CD pipeline + one properly managed server (plus a staging server) is what "professional" looks like for an internal company system with tens-to-hundreds of users. Kubernetes is for orgs running dozens of services with dedicated platform teams.

Estimated effort for the full plan in §6: roughly **10–14 weeks** of focused work, deliverable in independent phases that each leave the system better without breaking it.

---

2. System Inventory — What It Has

2.1 Stack

Layer	Technology	Assessment
Backend	Laravel 13, PHP 8.4	Current, industry standard ✅
Frontend	React 19 + TypeScript + Vite + Tailwind v4	Current, industry standard ✅
Database	PostgreSQL (SQLite for local dev)	Solid choice ✅
Auth	Sanctum bearer tokens + Google Authenticator TOTP 2FA	Works; token storage needs hardening (§4.4)
Cache / Queue / Session	Redis (configured in `.env.example`)	Configured but queues barely used (§4.2)
Real-time	Laravel Reverb installed **and** a hand-rolled `ws-server.js`	Duplicated — pick one (§4.2)
Performance	Laravel Octane + RoadRunner installed	Good instinct, not operationalized
PDF	dompdf (backend), jsPDF (frontend)	Works

2.2 Scale of the codebase

Metric	Count
Backend controllers	47 (grouped: Accounting, Procurement, Travel, Fleet, Inventory, Admin, Sales, Auth + top-level)
Eloquent models	59
Database migrations	140
Service classes	14 — split across **two** namespaces (`app/Http/Services/` and `app/Services/`)
API route file	573 lines, single `routes/api.php`
Backend tests	20 Feature test files → 126 tests, 416 assertions
Frontend source files	153 `.ts` / `.tsx` (~42,000 lines in `pages/` alone)
Frontend API modules	29 (one per domain, on a shared Axios client)
Frontend tests	**0**
Shared UI components	16 (`Button`, `Modal`, `Pagination`, `StatusBadge`, `ErrorBoundary`, etc.)
API contracts (OpenAPI)	13 modules covered in `api-contracts/` (partial coverage)

2.3 Functional modules (what it does)

- **Auth & Admin** — login, TOTP 2FA, forced password change, user management, hybrid RBAC (hardcoded roles + dynamic `role_permissions` table), audit logs, system settings, role/permission editor.
- **Procurement** — suppliers, purchase orders, job orders, work orders, document management with categories, supplier accreditation with public KYC submission.
- **Travel** — customers, passports, visas (cases + requirements + document checklists), passengers, KYC, agent tasks, legal documents, customer emails.
- **Sales** — quotations, fixed packages, checkout, the contract system (drafts → e-signature → finalization), commissions, customer portal (public token-scoped upload + signing).
- **Accounting** — chart of accounts, journal entries, ledger, billing/invoices, collections, liquidations, cash budget requests, reports, PayMongo payment integration.
- **Fleet / Logistics** — buses, trip tickets (with conflict detection), accreditations, PMS maintenance schedules, driver portal.
- **Inventory** — items, stock, PMS.
- **HR** — employees, payroll cycles, payslips, salaries, job applications with document upload, internships.
- **Cross-cutting** — in-app chat (WebSocket), notifications, dashboards per role, entity preview, dark mode.

This is a genuine ERP. The functional surface is the system's main asset — the refactor must preserve it.

---

3. What Works As Intended (verified)

3.1 Verified by the automated test suite (run 2026-07-04)

Result: 126 tests, 125 passed, 1 failed, 416 assertions, ~60s.

Modules with passing feature tests — these behave as their tests specify:

Area	Test file	Status
Authentication (login, 2FA, set-password)	`AuthTest`	✅ pass
User management + RBAC	`UserManagementTest`, `DriverAccessTest`	✅ pass
Audit logging	`AuditLogTest`	✅ pass
Suppliers	`SupplierTest`	✅ pass
Purchase orders	`PurchaseOrderTest`	✅ pass
Job orders	`JobOrderTest`	✅ pass
Work orders	`WorkOrderTest`	✅ pass
Auto-numbering / generation	`AutoGenerationTest`	✅ pass
Customers	`CustomerTest`	✅ pass
Passport cases + public visa upload	`PassportCaseTest`, `VisaPublicUploadTest`	✅ pass
Accounting	`AccountingTest`	✅ pass
Inventory	`InventoryTest`	✅ pass
Trip ticket conflict detection	`TripTicketConflictTest`	✅ pass
Bus schedule sync	`BusScheduleSyncTest`	✅ pass
HR application documents	`JobApplicationDocumentTest`	✅ pass
Notifications	`NotificationApiTest`	✅ pass
Payroll	`PayrollTest`	❌ **1 failure**

The one failure — fix or explain before anything else: `PayrollTest::test_admin_can_run_payroll_cycle` expects tax `2,604.17` on gross `28,000` but the system now computes `3,204.17`. Either the tax computation changed intentionally (→ update the test) or a regression was introduced on this branch (→ payroll is currently miscalculating tax by ₱600/cycle/employee). **This is money math; treat as priority zero.**

3.2 Verified by type-check / build

- `tsc -b` compiles with **one** error on this branch: unused variable `rule` in `frontend/src/pages/travel/VisaProcessing.tsx:574`. This blocks `npm run build`. One-line fix.

3.3 Security findings previously reported — verified FIXED in current code

The old QA reports in the repo root are partially stale. Re-checked against source today:

Finding	Status now
Login brute-force	✅ Fixed — `RateLimiter` 5/min per IP in `AuthController::login`
2FA code brute-force	✅ Fixed — rate limited
Passwords leaking into audit logs	✅ Fixed — all password variants excluded in `AuditLogger`
Dead 2FA middleware	✅ Fixed — `verify.2fa` applied at `routes/api.php:197`
Avatar upload memory DoS	✅ Fixed — 5 MB pre-decode size check
RCE via job-application upload	✅ Fixed — MIME whitelist, server-resolved extension, private storage
Super-admin route bypass / privilege escalation	✅ Fixed per remediation summary
Public upload endpoints unthrottled	✅ Fixed — `throttle:10,1` on all public KYC/portal/visa uploads

3.4 Working but unverified (no automated coverage)

These run in production use but have **no tests** — they "work" only anecdotally: Sales contracts & customer portal, quotations, fixed packages, commissions, billing/invoicing, collections, liquidations, cash budgets, reports, chat, dashboards, payslips, internships, legal documents, agent tasks, PMS, and essentially **the entire frontend**. Anything in this list can silently break in any merge. That is the single biggest quality risk in the system.

---

4. What's Missing or Below Professional Standard

4.1 Infrastructure & DevOps — the biggest gap (all absent)

Standard practice	Status	What it means
CI (automated tests on every push/PR)	❌ None (no `.github/`)	Nothing stops broken code reaching `main`. The payroll failure and TS error on this branch are exactly what CI catches.
CD (automated deployment)	❌ None	Deployment is manual and undocumented — unrepeatable, error-prone.
Docker	❌ None	"Works on my machine" setup; the committed `php83/` portable toolchain and `php83.zip` are a workaround for exactly the problem Docker solves.
Staging environment	❌ None	Changes go from laptop to users with no rehearsal.
Error monitoring (Sentry/Bugsnag)	❌ None	Production errors are invisible unless a user complains.
Uptime monitoring & alerting	❌ None	—
Database backups (automated, tested restore)	❌ Not documented anywhere	For an accounting + payroll system this is existential.
Secrets management	⚠️ `.env` only; seeders create accounts with `password123`	Seeded credentials must never reach production.
Log aggregation	⚠️ Local files, `LOG_LEVEL=debug`	Fine for dev; production needs daily rotation + shipped logs.
Queue workers as a managed service	❌ `queue:listen` in a dev script only	Emails/PDFs block web requests in production (§4.2).

4.2 Backend code quality

- **God controllers.** `TripTicketController` (838 lines), `ProcurementDocumentController` (789), `DashboardController` (713), `PublicRequestActionController` (633), `BillingController` (616). Business logic belongs in services; controllers should be thin. Only 14 service classes exist for 47 controllers, and they're split across two namespaces (`app/Http/Services/` vs `app/Services/`) — consolidate to one.
- **Almost nothing is queued.** No `app/Jobs/` directory; only 3 of 11 mailables/notifications implement `ShouldQueue`. Sending contract emails, generating PDFs, and outreach mail all block the HTTP request. Redis queueing is already configured — it's just unused.
- **Inconsistent API layer.** 14 API Resources and 12 FormRequests for 47 controllers — most endpoints validate inline and return raw model JSON (leaks any newly added column to every consumer). Only 19 controllers paginate; the rest return unbounded `->get()` lists that will degrade as data grows.
- **Rate limiting only on auth + public routes.** Authenticated API has no global throttle — one runaway dashboard poll can saturate the server for everyone.
- **No API versioning** (`/api/v1/...`) — every frontend/backend deploy must be in lockstep.

- **Closure routes in `api.php`** (e.g. public `/public/buses`) — no auth, no controller, can't be cached with `route:cache`.
- **Two real-time systems.** Laravel Reverb (installed, standard) and a hand-rolled `ws-server.js` relay that broadcasts every chat message to **all** connected clients with no authentication. Kill `ws-server.js`, use Reverb with private channels.
- **`routes/api.php` is a 573-line monolith** — split per module.
- **N+1 / index audit never done** — 140 migrations, no evidence of a query-performance pass.

4.3 Frontend code quality

- **Zero tests.** No Vitest, no React Testing Library, no Playwright. Nothing.
- **No code splitting.** `App.tsx` has zero `React.lazy` — all ~200 routes/pages compile into one bundle. A driver opening their trip page downloads the payroll module, the accounting module, and three PDF/Excel libraries.
- **God components.** `FixedPackages.tsx` (2,685 lines), `KycSubmission.tsx` (2,075), `CashBudgets.tsx` (1,675), `Users.tsx` (1,541)... 14 pages exceed 1,000 lines. These are unreviewable and unmodifiable-safely.
- **Two data-fetching patterns.** React Query is installed but used in only ~5 files; everything else is raw `axios` + `useEffect` + manual loading state — the source of stale-data and race-condition bugs, resolved by hand hundreds of times.
- **Redundant dependencies.** Two icon sets (`lucide-react` + `react-icons`), two notification systems (`sweetalert2` + `react-hot-toast`), two date tools. Each pair = inconsistent UX + bundle weight.
- **No design system.** 16 shared UI components, but pages largely hand-roll their own tables, filters, modals and forms — which is why the UI feels unpolished relative to your Figma. The Figma files (Micro Dashboard / Micro-Animations / Dashboard Flaws) should be translated into design tokens + a component library that pages consume (§6 Phase 3).
- **Forms inconsistent** — `react-hook-form` + `zod` are installed but most pages manage form state by hand with ad-hoc validation.

4.4 Security & auth (remaining items)

- **Bearer token in `localStorage`.** Any XSS = full account theft. The professional pattern for a same-domain SPA is Sanctum's cookie-based SPA mode (httpOnly cookies, immune to JS theft). Medium-effort migration, high value.
- **Seeded weak credentials** (`password123`, documented in `seeded_accounts.md`) — acceptable for dev, must be provably absent in production (seeder guards on `APP_ENV`).
- **No password policy / expiry / session-revocation UI.**
- **`APP_DEBUG=true` in the example env** — a production deploy that copies it leaks stack traces and env details. Add a deploy check.
- **Public closure endpoints** (`/public/buses`) expose fleet data unauthenticated by design — confirm that's intended, move to a controller with explicit field whitelist either way.

4.5 Repo & process hygiene

- **Committed junk:** `test.php` – `test7.php` (debug one-liners), `php83.zip`, `rr.exe`, `output.txt`, `test_output.txt`, `debug_*.json`, `diff.txt` / `diff_utf8.txt` (×2), `CustomerProfile_backup.tsx`, `busBlock.txt`, `toArray())` (a file literally named a syntax fragment), `localtunnel*.log`, plus the 99-file

`php83/` + `tools/` portable toolchain. 777 tracked files, roughly 15% junk.

- **Branch strategy:** personal branches (`Emman` , `Val` , `Greg` , `greg_Backend`) with direct pushes; no PRs, no reviews, no protected `main` . 311 commits with messages like "VISA REQUIREMENTS FIX".
- **Python "tests"** (`backend_tests.py` etc.) hit a live server with hardcoded credentials — not CI-compatible; port anything valuable into PHPUnit and delete.
- **Docs are contradictory** — root has 10+ overlapping reports/manuals, several stale (this audit supersedes the QA status claims; §3.3 re-verified them).

---

5. Professional Standards to Adopt — Plain-English Guide

5.1 The must-haves (non-negotiable for "industry level")

1. **Git workflow with PRs.** Protect `main` . All work on short-lived feature branches → Pull Request → CI green + 1 review → merge. This is the single highest-leverage change and costs nothing.
2. **CI — GitHub Actions.** On every PR, automatically: `pint --test` + `php artisan test` (Postgres service container) on backend; `eslint` + `tsc -b` + `vite build` on frontend. A red ❌ blocks merge. Today's payroll failure and TS error would have been caught at commit time.
3. **Docker for dev + deploy.** One `docker-compose.yml` (php-fpm/octane, postgres, redis, vite, nginx) replaces the entire `php83/` + `tools/` folder and guarantees every developer and the server run identical stacks.
4. **A real deployment pipeline + staging.** Two servers (or one server, two isolated envs): `staging` auto-deploys from `main` , `production` deploys on a tagged release. Zero-downtime deploys via Docker or a tool like Laravel Forge/Ploi/Envoyer.
5. **Error monitoring.** Sentry (free tier is enough) on both Laravel and React. You will discover bugs users never reported.
6. **Automated, restore-tested DB backups.** Nightly `pg_dump` to offsite storage (e.g. S3), with a documented, rehearsed restore. Payroll + accounting data makes this existential, not optional.

5.2 The should-haves

1. **Queue workers in production** (Supervisor or a Docker service running `queue:work`) + make all mail/PDF generation queued.
2. **Sanctum SPA cookie auth** replacing localStorage tokens.
3. **API versioning + finish OpenAPI coverage** — `api-contracts/` already exists for 13 modules; make it the enforced contract for all.
4. **Frontend test stack**: Vitest + React Testing Library for components/hooks; Playwright for ~10 E2E happy paths (login→2FA, create PO, checkout→contract→portal sign, run payroll).
5. **Design system**: extract tokens (color/type/spacing/radius/motion) from your Figma files into Tailwind theme config + a documented component library; pages consume components, never hand-roll.
6. **Observability**: request logging, slow-query log, uptime checks (UptimeRobot/BetterStack).

5.3 What you've heard about but should SKIP (honest sizing)

Buzzword	Verdict for JVD	Why
Kubernetes	❌ Skip	K8s solves "orchestrate 50 services across 30 machines with a platform team." You have 1 app, 1 database, ≤ a few hundred internal users. K8s would add weeks of learning and permanent operational drag for zero benefit. Docker Compose on one good VPS handles this scale with room to grow 10×.
Microservices	❌ Skip	The Laravel monolith is correct for your team size. Modular monolith (clean module boundaries inside one app) is the professional pattern here.
Terraform/IaC	⏸ Later	Worth it when you have >2 servers. A documented server-setup script is enough now.
Service mesh, Kafka, gRPC	❌ Skip	Not your problem domain.
Docker, CI/CD, staging, monitoring, backups	✅ Adopt	This is the actual professional baseline — see 5.1.

Rule of thumb: professionalism at your scale = *reproducibility* (Docker), *safety nets* (CI, tests, staging, backups), and *visibility* (monitoring). Not exotic infrastructure.

---

6. Implementation Plan

Phases are ordered so each one de-risks the next. Don't parallelize Phase 0/1 with feature work — they're small and everything else depends on them.

Phase 0 — Stop the bleeding (Week 1) · effort: ~2–3 days

1. Fix the payroll tax test failure (investigate whether code or test is wrong — money math).
2. Fix the `VisaProcessing.tsx` TS error so `npm run build` passes.
3. Repo cleanup: delete `test*.php`, `php83.zip`, `rr.exe`, diff/output/log/backup files; move `php83/` + `tools/` out of git (keep as a downloadable zip if needed); tighten `.gitignore`.
4. Branch policy: protect `main`; agree that all work goes feature-branch → PR → review; merge/retire the personal branches (`Emman`, `Val`, `Greg` have diverged — reconcile now, it only gets worse).
5. Consolidate root-level docs into `docs/`, mark stale reports as superseded.

Done when: `main` is protected, repo is clean, both build commands pass.

Phase 1 — Safety net: CI/CD + environments (Weeks 1–3) · effort: ~1.5 weeks

1. GitHub Actions: `backend.yml` (pint, phpunit vs Postgres service) + `frontend.yml` (eslint, tsc, build) required on PRs.
2. `docker-compose.yml` for local dev (app, postgres, redis, vite); `Dockerfile` for production (php-fpm or Octane).
3. Provision staging + production environments; auto-deploy staging from `main`, tagged deploys to production; document rollback.
4. Sentry on backend + frontend; uptime monitor on both environments.

5. Nightly `pg_dump` → offsite storage; perform and document one restore drill.
6. Production env hardening: `APP_DEBUG=false`, `APP_ENV=production` guard on all seeders with test credentials, rotate any real accounts still on `password123`.

Done when: a PR cannot merge red; a merge reaches staging with zero manual steps; a deliberate test error shows up in Sentry; a backup has been restored once.

Phase 2 — Backend hardening (Weeks 3–6)

1. **Queues**: create `app/Jobs/`, make every Mailable/Notification `ShouldQueue`, move PDF generation to jobs, run `queue:work` under Supervisor/Docker.
2. **Thin the god controllers** (worst five first: TripTicket, ProcurementDocument, Dashboard, PublicRequestAction, Billing) — extract to services; merge the two service namespaces into `app/Services/`.
3. **Standardize the API layer**: FormRequest for every write endpoint (also mitigates the known `validate()` nested-key gotcha), API Resource for every response, pagination on every list endpoint.
4. Global `throttle:api` on authenticated routes; split `routes/api.php` per module; replace closure routes with controllers; introduce `/api/v1` prefix.
5. Auth migration to Sanctum SPA cookie mode (kills localStorage token risk).
6. Performance pass: N+1 audit (enable `Model::shouldBeStrict()` in dev), add missing DB indexes, decide Octane on/off and configure properly.
7. Real-time consolidation: delete `ws-server.js`, move chat to Reverb private channels.
8. Backend test expansion for the untested money paths: billing, collections, liquidations, contracts, commissions, cash budgets. Target: every controller that touches money has a feature test.

Done when: no controller > ~300 lines, all mail/PDF async, all list endpoints paginated, money modules ≥ 80% covered, one WebSocket system.

Phase 3 — Frontend professionalization + Figma redesign (Weeks 6–10)

This is where the three Figma files come in. (Note: `.fig` binaries can't be read programmatically — export frames as PNG/dev-mode specs, or share the Figma link, when this phase starts.)

1. **Design tokens first**: colors, typography, spacing, radii, shadows, motion curves from Figma → Tailwind v4 theme. One source of truth.
2. **Component library**: rebuild/extend `components/ui/` to match Figma (Button, Input, Select, Table with sorting/filtering/pagination, Modal, Drawer, Toast, Card, Stat, Skeleton, EmptyState) with `framer-motion` micro-animations per the Micro-Animations file. Standardize on **one** icon set (lucide) and **one** toast system (drop sweetalert2 or wrap it away).
3. **Data layer**: React Query everywhere — one `useQuery` / `useMutation` hook per API module; delete hand-rolled `useEffect` fetching as pages are touched.
4. **Code splitting**: `React.lazy` per route + module-level chunks; lazy-load exceljs/jspdf only where used. Target initial bundle < 300 KB gz.
5. **Forms**: `react-hook-form` + `zod` schema per form, shared `<Form>` components.
6. **Page-by-page migration** in module order of business value (Dashboards → Sales → Accounting → Travel → the rest), splitting each god page into `components/<module>/` pieces as it's redesigned. Address the specific issues in your "Dashboard Flaws" file during the Dashboards pass.

7. **Auth UX**: polished login/2FA/set-password flow per Figma (this is the first screen every employee sees daily).

Done when: every page uses the component library, no page > ~500 lines, bundle is split, one icon/toast/date library each.

Phase 4 — Test depth & performance proof (Weeks 10–12)

1. Vitest + RTL: cover the component library and critical hooks.
2. Playwright E2E: ~10 business-critical happy paths, run in CI nightly and pre-release.
3. Load test (k6): simulate expected concurrent company usage against staging; fix what falls over (this is where "handle many users" is proven, not assumed).
4. Coverage gates in CI (backend line coverage ratchet, no-new-untested-controller rule).

Phase 5 — Scale & polish (ongoing)

- File storage → S3-compatible (needed the day you run 2 app servers; also fixes backup of uploads).
- Redis response/query caching for dashboards and reports.
- Log aggregation, slow-query dashboards.
- Only if user count truly grows: add a second app server behind a load balancer — Docker makes this trivial; still no Kubernetes.

Sequencing rules

- Phases 0–1 before anything else. They are cheap and everything depends on them.
- Phase 2 and 3 can overlap if backend/frontend people are different, **after** CI exists.
- Every phase merges through PRs in small slices — never a "big bang" branch.

---

7. Priority Cheat-Sheet

#	Action	Impact	Effort
1	Fix payroll tax test failure	🔴 money correctness	hours
2	Protect `main` + PR workflow	🔴 stops future regressions	hours
3	CI pipeline	🔴	1–2 days
4	Repo cleanup	🟠	half day
5	Docker dev environment	🟠	2–3 days
6	Staging + deploy pipeline + Sentry + backups	🔴 operational survival	1 week
7	Queue all mail/PDF	🟠 perceived speed	2–3 days
8	Sanctum cookie auth	🟠 security	2–3 days
9	Design system from Figma	🟢 the "polish" you want	2 weeks
10	React Query + code splitting + page refactor	🟢 UX + maintainability	rolling
11	Money-module backend tests	🔴	rolling
12	E2E + load tests	🟢 proof of scale	1 week

Generated by full-codebase scan + live test run on branch `Emman`. Supersedes the status claims in `QA-IMPROVEMENTS.md` / `QA-REMEDIATION-SUMMARY.md` where they conflict (all previously-reported security criticals were re-verified as fixed in current code — see §3.3).